How Our Knowledge Engine Works

MVP approach, innovation and architecture · Nov 8, 2026 · @Arnav

The problem and our answer

We built an engine that reads whatever documents a company drops in, keeps one always-current knowledge graph, and answers questions with exact citations.

Companies keep their rules in SOPs, spreadsheets, slide decks, emails and chats, and those rules change. In our first test case, an accounts-payable team at a fictional company called Meridian Shared Services, 12.7% of answers given over chat were outdated and the median wait for an answer was 33 minutes. People followed old rules because nobody could tell which document was current.

The product has to work for any company, not just this one. Meridian is our first customer and our test dataset, not the product itself.

Our approach: a generic core with industry plug-ins

The engine has two layers: a core that knows nothing about any industry, and plug-ins that add industry-specific logic.

- **Generic core.** Takes in any file or message, extracts entities and facts, keeps them in a knowledge graph, resolves conflicts, and answers with citations. It never mentions Meridian, tax or accounts payable.
- **AP plug-in.** Holds what only makes sense for accounts payable: six calculators (price tolerance, quantity tolerance, TDS, approvals, MSME payment dates, e-invoice), who-to-ask routing, and an invoice picker. The calculators read their thresholds from the graph, so nothing is hard-coded.
- **Safety net.** A hand-written schema for the AP case sits beside the engine. One switch (`SCHEMA_MODE=baseline`) runs on it, so the demo cannot be blocked if automatic schema learning stumbles, and the same switch lets us measure how close the automatic version gets.

The graph follows the data model of the Zep research paper (the Graphiti engine): raw inputs called episodes, entities, and fact edges that are closed instead of deleted. We built this on Postgres rather than using the Graphiti library, because we needed three things it does not give us: source authority deciding which fact wins, typed values that calculators can compare, and owner review before anything goes live.

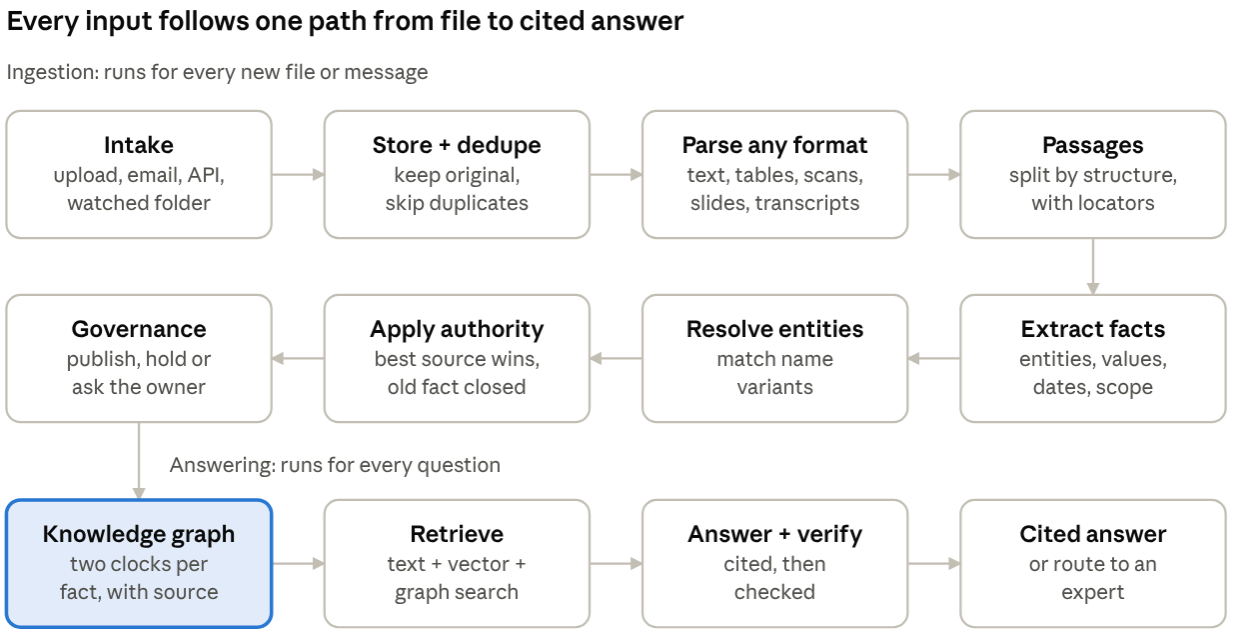
What we are innovating on

Four ideas set this apart from a standard document chatbot.

1. **Every fact has two clocks.** Each fact records when it was true in the world and when our system learned it. So we can answer "what was the rule on 1 March?" and also "what would the engine have told someone on 10 November?". Facts are closed off, never deleted.
2. **The best source wins, not the newest.** A stale FAQ ingested today never overrides the current SOP. The engine ranks sources by type, version and status, and the owner confirms the ranking.
3. **Any format, no setup.** The engine reads a layout it has never seen, learns the company's topics and parameters on its own (the owner reviews them once), and works from the first dump. We prove it with a second, unrelated company's documents and no code changes.
4. **It only answers from the knowledge base, and escalates the rest.** Every sentence must point to a source passage. If the documents don't cover a question, the engine says so and sends it to the right expert. The expert's reply goes through owner review before it becomes a rule.

End to end: from a dropped file to a cited answer

Every input, whether the first bulk upload or the hundredth email, goes through the same pipeline, and a background worker runs it so large uploads never block the app.

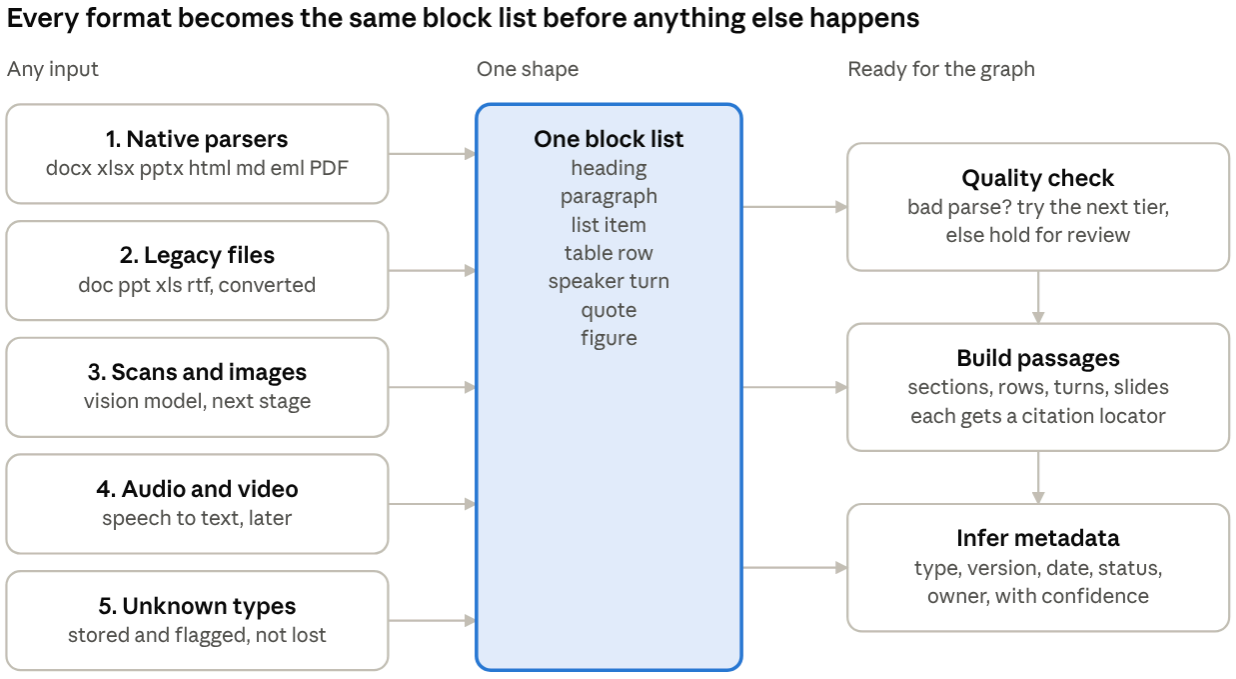


end-to-end flow · 12 steps in two stages

The top two rows build the graph. The bottom row answers questions from it. A change to a rule closes the old fact and opens the new one in a single step, so a reader never sees a half-updated state.

Handling any document in any format

Nothing after parsing knows what format or layout a document had, so a new layout never needs new code.



format handling · 5 parser tiers into one block list

Structure is read from real signals first (styles, font sizes, slide titles, email headers). Numbering such as 4.2 is only a hint. If the outline is still flat, a small AI step assigns heading levels. File type is detected from the file contents, not its extension. The original file is always kept, so a citation can open it with the passage highlighted.

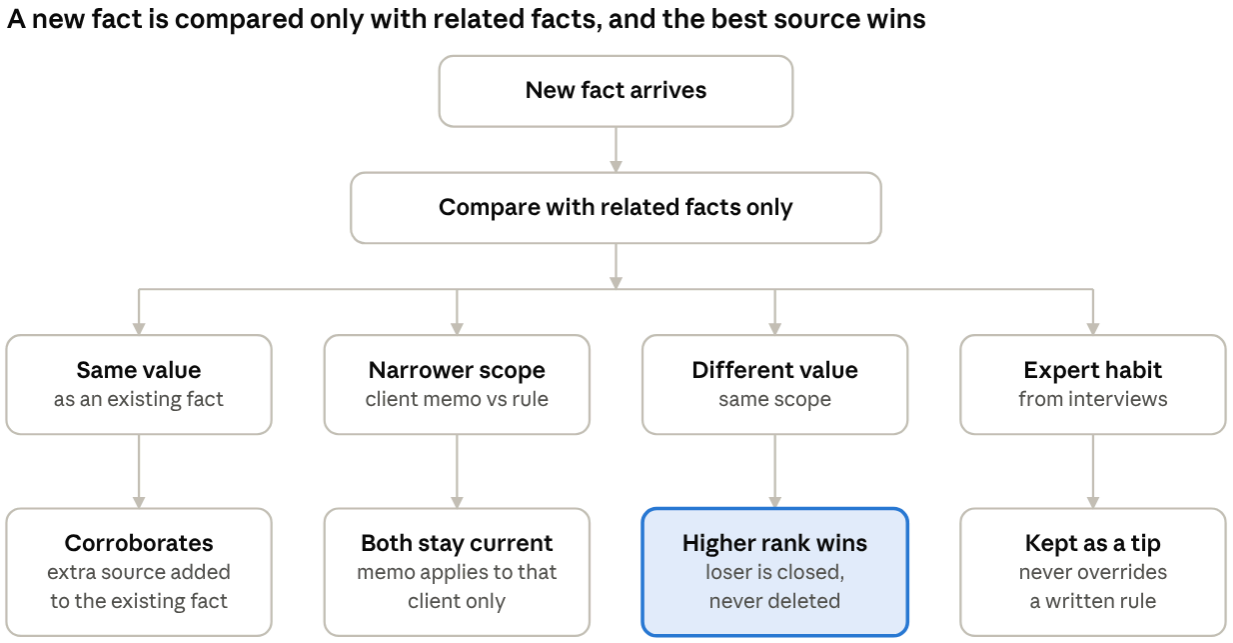
How the graph stays correct over time

Each fact carries four timestamps, on two clocks, and nothing is ever overwritten.

Timestamp	Clock	Meaning
valid_from	Valid time	When the fact became true in the world
valid_to	Valid time	When it stopped being true
recorded_at	Transaction time	When our system learned it

Timestamp	Clock	Meaning
retracted_at	Transaction time	When our system stopped believing it

Valid time answers "what was the rule on 1 March?". Transaction time answers "what would the engine have told someone on 10 November?", which audits and answer recall need.



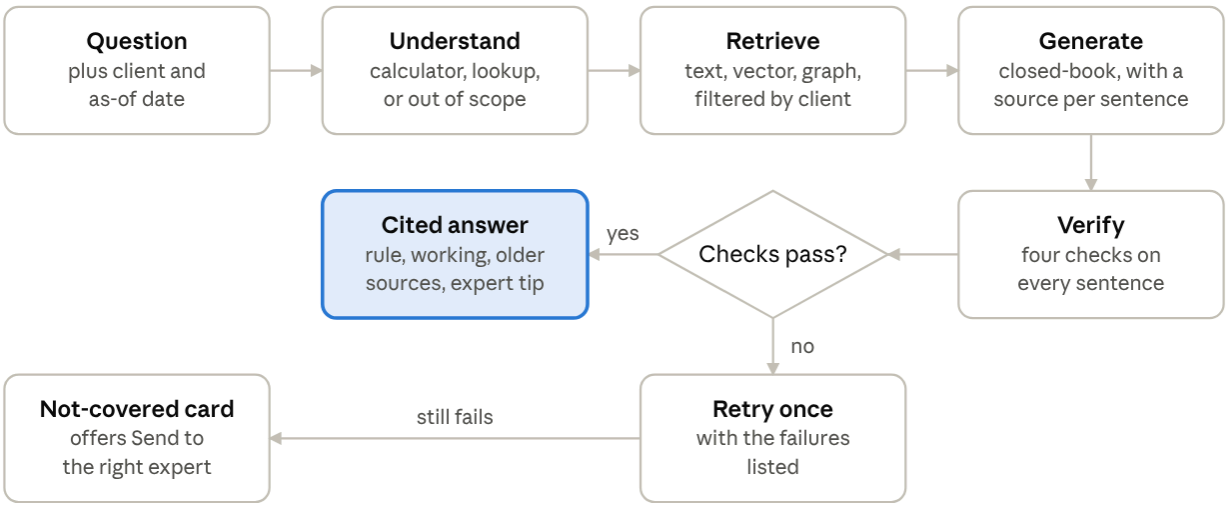
conflict resolution · 4 cases for a new fact

A document ingested later does not automatically win. A newer FAQ from a lower-ranked source never overrides the current SOP. A client memo can override a general rule for that client only.

How an answer is built and checked

Retrieval finds the evidence, the model only writes from it, and a separate verifier checks the result before it is shown.

An answer is built from the graph, then checked before anyone sees it



A retry that passes is shown as normal. Processors never see text the verifier did not confirm.

answer pipeline · build, verify, then show or hand off

- **Retrieval** mixes keyword search, vector search and a walk through the graph, then reranks. It filters by the asker's client and by the as-of date, so an old question gets the old rule.
- **Numbers** come from the graph or a calculator, never from the model's own arithmetic.
- **The answer card** shows the rule with numbered citations, the working for calculations, an "older sources" line naming what was superseded and when, and any approved expert tip.
- **The verifier** checks four things: every cited passage was actually retrieved and contains the quote, every number appears in a source or calculator output, no superseded value is presented as current, and each sentence is supported by its citation.

What is different about our retrieval

Most chatbots search text and hope the right chunk comes back. We search text and the graph together, and filter by time and client before the model sees anything.

- **Hybrid search, merged fairly.** Keyword search, vector search and the graph each return candidates. Reciprocal rank fusion merges them, a reranker re-scores the top 20, and the best 6 passages go forward.
- **Time-aware.** Retrieval filters by the as-of date. A question about 1 March gets the rule in force on 1 March, and superseded passages are fetched separately for the "older sources" line, never as main evidence.

- **Scope-aware.** Results are limited to general rules plus the asker's client, so one client's memo never leaks into another client's answer.
- **Graph first for facts.** The question is linked to entities (client, vendor, policy), and their current facts are pulled straight from the graph, then expanded one or two hops. A rule is read as a stored value, not re-guessed from prose each time.
- **Passages built around structure.** Chunks follow sections, table rows, transcript turns and slides, and each carries a header (document, version, date, status). The same text from an old and a new version cannot be confused.
- **Right tool per question.** A router sends calculations to calculators, lookups to retrieval, and out-of-scope questions to a refusal. Only lookups depend on search quality.
- **Cheap on repeat.** Embeddings, extractions and checks are cached, so the same input never pays twice.

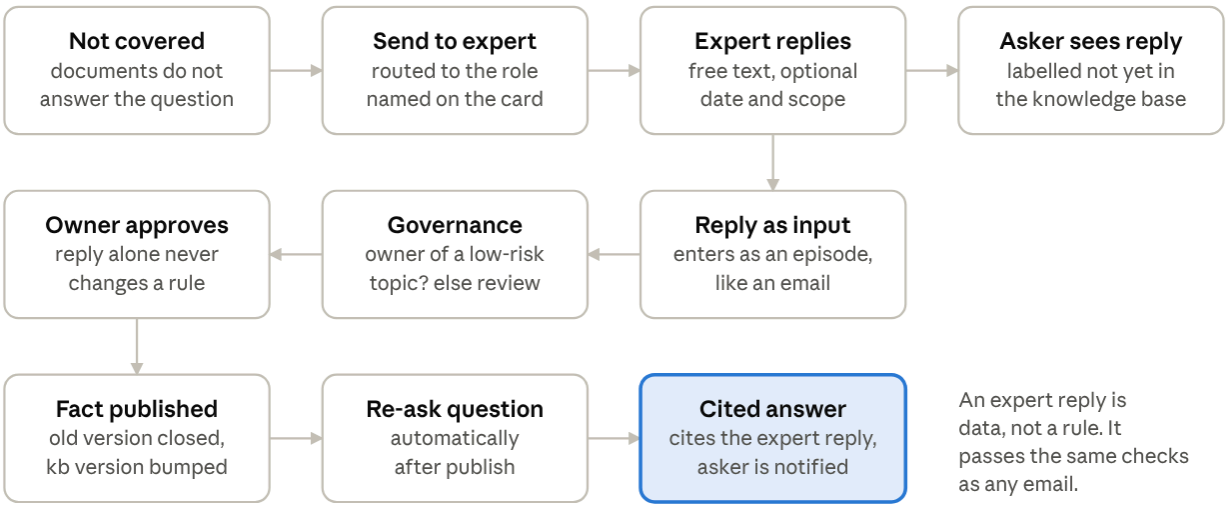
This design is built and specified. We are measuring it with an ablation that compares plain vector search, the whole corpus in context, the corpus plus the graph, and the full system, so the gain from each layer is visible.

Guardrails and expert escalation

The chatbot answers only from the knowledge base, through layers that do not depend on the model behaving, and what it cannot answer goes to a person.

1. **Scope guard.** General knowledge, chit-chat and advice beyond the documents are refused before any answer is written.
2. **Closed-book writing.** The model sees only retrieved passages, current facts and calculator output. Every sentence must carry a passage id or it is rejected.
3. **Retrieval gate.** If the best evidence is weak, the engine abstains instead of guessing.
4. **Verifier.** A failed check withholds the draft from processors. They see the not-covered card with an escalate button; only the process owner sees the draft.
5. **Deterministic numbers.** Calculators and graph values supply every figure.
6. **Injection resistance.** Text inside emails, uploads and expert replies is treated as data, never as instructions.
7. **Audit trail.** Every answer, its citations and its verifier result are logged, and a "Report a wrong answer" button on each card opens an owner review item.

Expert replies are reviewed before they become rules



expert escalation · from unanswered question to cited answer

The reply reaches the asker straight away, labelled as not yet part of the knowledge base. It only becomes a rule if the sender is the registered owner of a low-risk topic (then it is provisional) or the process owner approves it. Spoofed or high-risk changes, such as a change to bank details, are always held for review.

Tech stack

One Postgres database holds the graph, the vectors and the job queue, which keeps the system small enough to build and run quickly.

Layer	What we use	Why
API	Python 3.11, FastAPI, Pydantic v2, SQLAlchemy 2	Typed, fast to build, auto-generated API docs
Database	Postgres 16 with pgvector, pg_trgm and btree_gist	Graph tables, vector search, fuzzy name matching and time-range indexes in one place
Background jobs	Postgres job queue (SELECT … FOR UPDATE SKIP LOCKED) and a worker process	Large uploads never block the app; failed jobs retry

Layer	What we use	Why
Search	Postgres full-text plus vector search, merged by reciprocal rank fusion; fastembed with BGE small embeddings and a BGE reranker, run on CPU	No extra search service
AI models	Anthropic Claude through one gateway: Haiku for triage, routing and structure repair; Sonnet for schema learning, extraction, answering, verification and vision	Cheap model for simple steps, strong model for the hard ones
Document parsing	python-docx, openpyxl and pandas, python-pptx, PyMuPDF, BeautifulSoup, libmagic, LibreOffice (headless) for old Office files	One parser per family, all producing the same block list
Web app	Next.js 14, TypeScript, Tailwind CSS, lucide-react; fonts Hanken Grotesk, Inter and JetBrains Mono	Light rose answer-engine design, calls the API only
Testing	pytest, ruff, eslint, tsc, Playwright	Unit, acceptance and screenshot checks
Deployment	Docker services on Render for the API and worker, Vercel for the web app, Neon for Postgres	One public URL

Three design rules sit across the stack:

- **One gateway for every model call.** It hides personal data such as PAN and bank numbers before anything is sent, logs each call, and caches results so a rebuild gives the same graph.
- **One repository layer.** All graph reads and writes go through `app/store/`, and every table has a tenant id, so the storage engine can change later and customers stay separate.
- **Tests keep the core generic.** One test fails the build if the core mentions Meridian or tax terms, and another fails it if app code touches the answer key.

Built to scale

The limit on scale is the number of AI calls per document, not the database, so most of our design reduces and reuses those calls.

Pressure	What we do about it
Many documents arriving	Ingestion runs in background workers on a Postgres queue. Workers can be added, failed jobs retry, and a killed worker leaves the same final graph
Cost of AI extraction	Triage drops noise before extraction. A new version of a document re-extracts only changed passages. Every result is cached by input, so rebuilds are free
Graph growing large	A new fact is compared only with related facts (same subject and attribute, or nearest neighbours), never the whole graph
Slow queries	Range indexes (GiST) on both clocks, vector indexes (HNSW) on every embedding, full-text and fuzzy-name indexes
Many customers	A tenant id on every table from the start, so partitioning by customer is a later change, not a rewrite
Changing the database later	Every graph read and write goes through one repository layer, so the storage engine can be swapped
Many users asking at once	Reads use a consistent snapshot, so a reader mid-update sees the old or the new state, never a mix

We have not yet run this at large scale. The plan includes a test with 20 concurrent users and a 50-document dump timed end to end, and it sets a target of answers in under 10 seconds at the median. If one customer passes roughly 10 million facts and multi-hop questions dominate, we would move to partitioned tables or a dedicated graph store, and the repository layer is what makes that possible.

How we prove it works, and what comes next

We test two claims: that it solves the Meridian case, and that it is a generic engine and not an accounts-payable tool. The figures below are the targets the MVP must hit, not results yet.

What we check	MVP target
Scenario calculations (92 questions)	92 of 92 correct, or every miss explained
Citation accuracy	At least 85%
Outdated answers given	Under 5%
Known conflicts found	At least 12 of 15
Unsupported answers shown to processors	Zero when supporting passages are removed
Expert escalation, end to end	Passes (test T11)
Same SOP in other formats (PDF, Markdown, HTML, XLSX, email text)	Facts match the original at 0.9 or better
A second, unrelated company's documents, no code changes	10 of 10 questions answered with citations
Replay of documents in date order	Every as-of question returns the right value

The other tests that protect honesty: the answer key is never read by app code, the evaluation set is split into tuning and held-out parts, and every headline number is reported for both the learned schema and the hand-written one so the cost of generality is visible.

After the MVP:

- Scanned PDFs and images through a vision model, PowerPoint, and a watched folder.
- Change notices when a rule a reader relied on is replaced, with acknowledgement counts.
- Expert interview capture for tacit knowledge, and audio or video transcription.
- Live connectors for Drive, SharePoint, Gmail and Slack, plus login, per-document access control and multiple companies on one deployment.